



Department of Computer Science and Engineering
University of Dhaka

Implementation of Client-Server Communication using Socket Programming

CSE 3111: Computer Networking Lab

Batch: 29/3rd Year 1st Semester 2024

Prepared by:

Mehedi Hasan (22) and Biplob Pal (50)

September 2025

Contents

1	Introduction	2
2	Objectives	2
3	Design Details	2
4	Implementation	3
4.1	Server.java Code	3
4.2	Client.java Code	5
4.3	Screenshots	5
5	Result Analysis	5
6	Discussion	6

1 Introduction

Socket programming is a fundamental technique in computer networking that enables communication between two processes over a network. It provides a standardized interface for applications to send and receive data using protocols like TCP or UDP. In Java, socket programming involves creating sockets—endpoints for communication—that can be connection-oriented (using TCP for reliable, ordered delivery) or connectionless (using UDP for faster but less reliable transmission). Sockets abstract the underlying network details, allowing developers to focus on application logic while handling data transmission via input and output streams.

Multi-threaded socket programming extends basic socket programming by incorporating threads to manage concurrent connections. In a basic socket setup, a server can typically handle only one client at a time, leading to blocking behavior where subsequent clients must wait. Multi-threading addresses this by assigning each client connection to a separate thread, enabling the server to process multiple requests simultaneously without interrupting ongoing communications. This is achieved by extending the Thread class or implementing Runnable, where each thread manages its own input/output streams for independent client-server interactions.

The necessity of socket programming and multi-threading for designing a chat application is evident in real-time communication scenarios. A chat application requires reliable, bidirectional data exchange between clients and a server. Basic sockets provide the foundation for establishing connections and transferring messages, while multi-threading ensures scalability—allowing multiple users to chat concurrently without performance degradation. Without multi-threading, the application would be limited to single-user interactions, making it unsuitable for group or multi-user chats. This combination enables features like instant messaging, where the server can relay or respond to messages from various clients in parallel, mimicking real-world applications like WhatsApp or IRC servers.

2 Objectives

- Implement a multi-threaded server that handles connections from multiple clients simultaneously, enabling concurrent chat sessions.
- Develop a fully functional chat application where clients can send single or multiple sentences to the server, with independent server replies.
- Incorporate termination conditions for both client and server sides to gracefully end communication sessions.

3 Design Details

The chat application is designed using Java's socket programming with multi-threading to support multiple clients. The system consists of a server that listens for incoming connections and spawns a new thread for each client, ensuring independent communication channels. Clients connect to the server, send messages, and receive replies. The termination condition is triggered when a client sends the message "Exit", closing the

connection for that client. The server continues running to accept new connections.

Step-by-step implementation process:

1. **Server Initialization:** Create a `ServerSocket` on port 5000 to listen for client connections in an infinite loop.
2. **Accept Client Connection:** Accept incoming connection requests to create a plain `Socket` for communication.
3. **Create Input/Output Streams:** Extract `DataInputStream` (for reading) and `DataOutputStream` (for writing) from the socket.
4. **Spawn Client Handler Thread:** Instantiate a `ClientHandler` object (extending `Thread`), passing the socket and streams, and start the thread.
5. **Client Handler Logic:** In the thread's `run()` method, loop to read client messages using `readUTF()`. If the message is "Exit", close the connection. Otherwise, print the message to the server console, prompt for a reply via `System.in`, and send it using `writeUTF()`.
6. **Client Side:** Create a `Socket` connecting to the server's IP and port. Set up `DataInputStream` and `DataOutputStream`. Loop to read user input, send it using `writeUTF()`, and read the server's reply using `readUTF()`. Exit on "Exit".
7. **Error Handling:** Use try-catch blocks for `IOExceptions` to handle connection issues gracefully.
8. **Termination:** Client terminates on "Exit". Server runs indefinitely but closes individual client sockets upon termination.

This design overcomes the limitations of basic socket programming by using threads to prevent blocking, allowing the server to handle multiple clients without queuing.

4 Implementation

The implementation is done in Java, with two main files: `Server.java` (containing the `Server` class and `ClientHandler` inner class) and `Client.java`.

4.1 Server.java Code

```
1 import java.io.*;
2 import java.net.*;
3 import java.util.*;
4
5 public class Server {
6     private static final int PORT = 5000;
7
8     public static void main(String[] args) {
9         try (ServerSocket serverSocket = new ServerSocket(PORT)) {
10             System.out.println("Server started on port " +
11                                PORT);
```

```
11         while (true) {
12             Socket clientSocket = serverSocket.accept();
13             System.out.println("New client connected: " +
clientSocket);
14             new ClientHandler(clientSocket).start();
15         }
16     } catch (IOException e) {
17         e.printStackTrace();
18     }
19 }
20 }
21
22 class ClientHandler extends Thread {
23     private final Socket socket;
24     private DataInputStream dis;
25     private DataOutputStream dos;
26
27     public ClientHandler(Socket socket) {
28         this.socket = socket;
29         try {
30             dis = new DataInputStream(socket.getInputStream())
;
31             dos = new DataOutputStream(socket.getOutputStream
());
32         } catch (IOException e) {
33             e.printStackTrace();
34         }
35     }
36
37     @Override
38     public void run() {
39         String received;
40         Scanner scanner = new Scanner(System.in);
41         while (true) {
42             try {
43                 received = dis.readUTF();
44                 if (received.equalsIgnoreCase("Exit")) {
45                     System.out.println("Client " + socket + "
exited.");
46                     break;
47                 }
48                 System.out.println("Client " + socket + ": " +
received);
49                 System.out.print("Server reply to " + socket +
": ");
50                 String reply = scanner.nextLine();
51                 dos.writeUTF(reply);
52             } catch (IOException e) {
53                 System.out.println("Connection error with " +
socket);
54                 break;

```

```
55     }
56 }
57 try {
58     socket.close();
59 } catch (IOException e) {
60     e.printStackTrace();
61 }
62 }
63 }
```

4.2 Client.java Code

```
1  import java.io.*;
2  import java.net.*;
3  import java.util.*;
4
5  public class Client {
6      private static final String HOST = "localhost";
7      private static final int PORT = 5000;
8
9      public static void main(String[] args) {
10         try (Socket socket = new Socket(HOST, PORT)) {
11             DataOutputStream dos = new DataOutputStream(socket
12                 .getOutputStream());
13             DataInputStream dis = new DataInputStream(socket.
14                 getInputStream());
15             Scanner scanner = new Scanner(System.in);
16             System.out.println("Connected to server. Type '
17             Exit' to quit.");
18             while (true) {
19                 System.out.print("Client message: ");
20                 String message = scanner.nextLine();
21                 dos.writeUTF(message);
22                 if (message.equalsIgnoreCase("Exit")) {
23                     break;
24                 }
25                 String reply = dis.readUTF();
26                 System.out.println("Server: " + reply);
27             }
28         } catch (IOException e) {
29             e.printStackTrace();
30         }
31     }
32 }
```

4.3 Screenshots

5 Result Analysis

The application was tested with multiple clients connecting to the server simultaneously.

```

PS C:\Users\hp\Desktop\networking> java server
Server started on port 5000
New client connected: Socket[addr=/127.0.0.1,port=60190,localport=5000]
Client Socket[addr=/127.0.0.1,port=60190,localport=5000]: hello server
Server reply to Socket[addr=/127.0.0.1,port=60190,localport=5000]: hwlllo client
Client Socket[addr=/127.0.0.1,port=60190,localport=5000]: ki koro
Server reply to Socket[addr=/127.0.0.1,port=60190,localport=5000]: bese achi
Client Socket[addr=/127.0.0.1,port=60190,localport=5000]: khawa dawa hoiche
Server reply to Socket[addr=/127.0.0.1,port=60190,localport=5000]: na kori nai tumi?
Client Socket[addr=/127.0.0.1,port=60190,localport=5000]: amio kori nai
Server reply to Socket[addr=/127.0.0.1,port=60190,localport=5000]: exit
Client Socket[addr=/127.0.0.1,port=60190,localport=5000] exited.

```

Figure 1: Server Side Console

```

PS C:\Users\hp\Desktop\networking> java Client
Server: hwlllo client
Client message: ki koro
Server: bese achi
Client message: khawa dawa hoiche
Server: na kori nai tumi?
Client message: amio kori nai
Server: exit
Client message: Exit
PS C:\Users\hp\Desktop\networking>

```

Figure 2: Client Side Console

Example Output (Server Side):

```

Server started on port 5000
New client connected: Socket[addr=/127.0.0.1,port=54321,localport=5000]
New client connected: Socket[addr=/127.0.0.1,port=54322,localport=5000]
Client Socket[addr=/127.0.0.1,port=54321,localport=5000]: Hello from Client 1
Server reply to Socket[addr=/127.0.0.1,port=54321,localport=5000]: Hi Client 1
Client Socket[addr=/127.0.0.1,port=54322,localport=5000]: Hi from Client 2
Server reply to Socket[addr=/127.0.0.1,port=54322,localport=5000]: Hello Client 2
Client Socket[addr=/127.0.0.1,port=54321,localport=5000]: Exit
Client Socket[addr=/127.0.0.1,port=54321,localport=5000] exited.

```

Example Output (Client 1 Side):

```

Connected to server. Type 'Exit' to quit.
Client message: Hello from Client 1
Server: Hi Client 1
Client message: Exit

```

Description: The server handles multiple clients concurrently, printing messages from each and allowing independent replies. Termination occurs smoothly on "Exit". If multiple messages are sent quickly, threads ensure they are processed without loss, though console input for replies requires careful sequencing by the server operator.

6 Discussion

Basic socket programming allows simple client-server communication but suffers from significant drawbacks, such as the inability to handle multiple clients concurrently. In a

non-threaded setup, the server processes one client at a time, causing others to block or fail to connect, leading to poor scalability and responsiveness. Additionally, if a client sends multiple messages without waiting for replies, the server may only receive the first one due to buffering issues, resulting in message loss.

Multi-threaded socket programming overcomes these by dedicating a thread per client, enabling parallel processing. This ensures the server remains available for new connections while handling ongoing chats, preventing blocking and message queuing problems. For instance, in our implementation, each `ClientHandler` thread manages its own streams, allowing independent communication without interference.

Through this lab, we learned the practical application of threads in networking, including synchronization challenges (e.g., shared console input) and the importance of exception handling for robust connections. Difficulties faced included managing interleaved console inputs on the server side, which could lead to mismatched replies in high-traffic scenarios—potentially resolvable with a GUI or queue-based input system. Overall, this enhanced our understanding of concurrent programming for real-time applications.